

Assignment 3

Name : Nazibul Islam Nabil

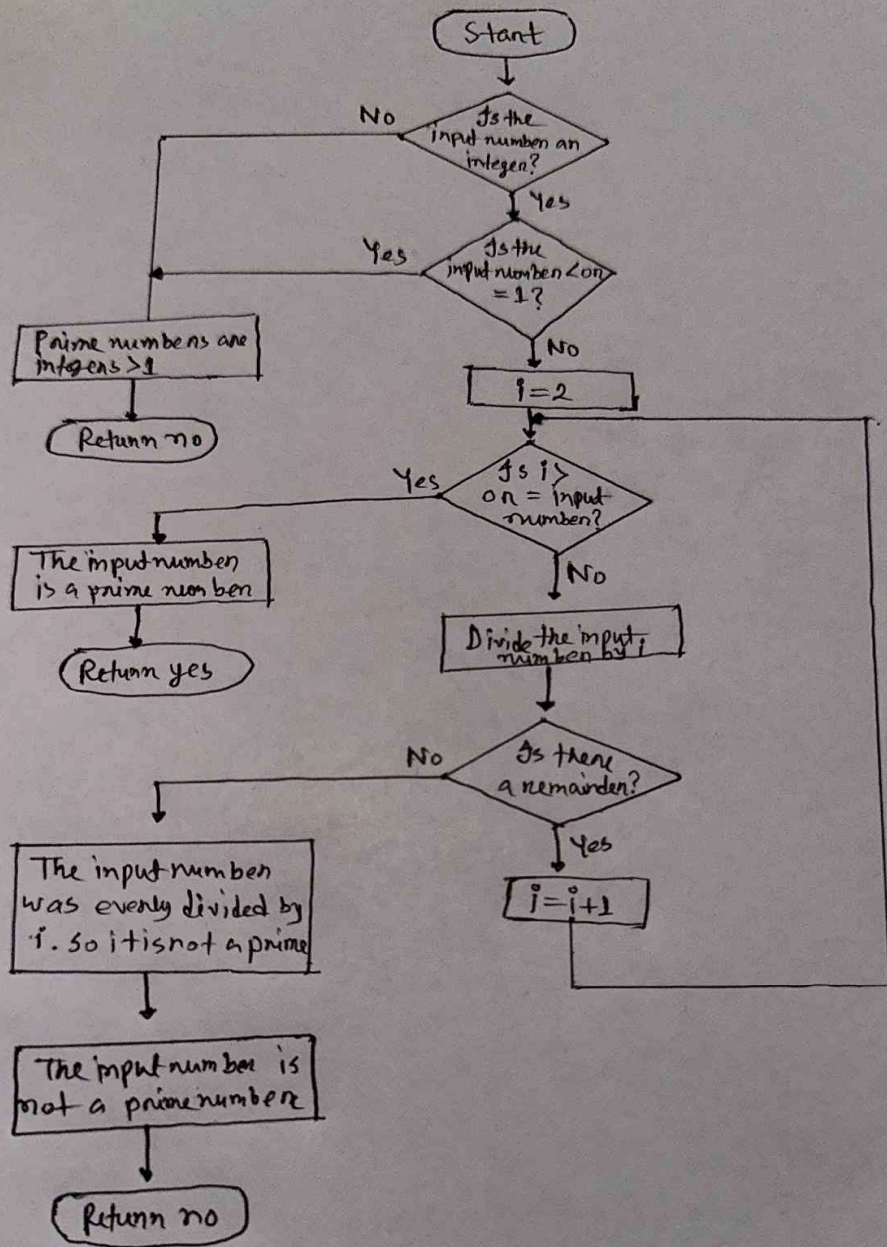
NSU ID : 2222456642

Course : CSE331

Section : 11

Date : 23-08-2025

a



b

```
#include <stdio.h>

int is_prime(int n) {
    if (n <= 1)
        return 0;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0)
            return 0;
    }
    return 1;
}

int main() {
    int integers[] = {4, 7, 11, 14, 16, 21, 23, 27, 29, 38, 39, 41, 47, 49, 53, 55, 69, 97, 99};
    int n = sizeof(integers) / sizeof(integers[0]);
    printf("Prime numbers:\n");
    for (int i = 0; i < n; i++) {
        if (is_prime(integers[i])) {
            printf("%d ", integers[i]);
        }
    }
    return 0;
}
```

c

.data

integers: .word 4, 7, 11, 14, 16, 21, 23, 27, 29, 38, 39, 41, 47, 49, 53, 55, 69, 97, 99

n: .word 19

prime_flag: .space 76

.text

.global _start

_start:

ldr r0, =integers

ldr r1, =n

ldr r1, [r1]

mov r2, #0

check_prime:

ldr r3, [r0, r2, LSL #2]

bl is_prime

cmp r0, #1

beq mark_prime

add r2, r2, #1

cmp r2, r1

blt check_prime

display_primes:

mov r0, #0

ldr r1, =msg

ldr r2, =msg_len

mov r7, #4

svc 0

_exit:

mov r7, #1

svc 0

is_prime:

mov r4, #2

cmp r0, #1

ble prime_exit

cmp r0, #2

beq prime_exit

check_divisor:

mov r5, r0

sdiv r5, r5, r4

mul r6, r5, r4

cmp r6, r0

beq prime_exit

add r4, r4, #1

mov r5, #0

cmp r4, r0

blt check_divisor

prime_exit:

```
moveq r0, #1
movne r0, #0
bx lr
mark_prime:
ldr r5, =prime_flag
ldr r6, =msg_len
ldr r7, [r5, r6]
add r7, r7, #1
str r7, [r5, r6]
ldr r7, =msg_len
ldr r8, [r5, r7]
cmp r8, r3
bge continue_check
str r3, [r5, r7]
continue_check:
add r6, r6, #4
mov r7, #0
str r7, [r5, r6]
b check_prime
.data
msg: .asciz "Prime numbers:\n"
msg_len: .word 25
```

d

.data

integers: .word 4, 7, 11, 14, 16, 21, 23, 27, 29, 38, 39, 41, 47, 49, 53, 55, 69, 97, 99

n: .word 19

prime_flag: .space 76

.text

.global _start

_start:

ldr r0, =integers

ldr r1, =n

ldr r1, [r1]

mov r2, #0

check_prime:

ldr r3, [r0, r2, LSL #2]

bl is_prime

cmp r0, #1

beq mark_prime

add r2, r2, #1

cmp r2, r1

blt check_prime

display_primes:

mov r0, #0

e

.data

integers: .word 4, 7, 11, 14, 16, 21, 23, 27, 29, 38, 39, 41, 47, 49, 53, 55, 69, 97, 99

n: .word 19

prime_flag: .space 76

.text

.global _start

_start:

ldr r0, =integers ; 0xE59F0000

ldr r1, =n ; 0xE59F1000

ldr r1, [r1] ; 0xE5911000

mov r2, #0 ; 0xE3A02000

check_prime:

ldr r3, [r0, r2, LSL #2] ; 0xE7903000

bl is_prime ; 0xE12FFF1E

cmp r0, #1 ; 0xE3500001

beq mark_prime ; 0x0A000001

add r2, r2, #1 ; 0xE0822001

cmp r2, r1 ; 0xE1520001

blt check_prime ; 0xBAFFFFFF8

display_primes:

mov r0, #0 ; 0xE3A00000

ldr r1, =msg ; 0xE59F1010

ldr r2, =msg_len	; 0xE59F2014
mov r7, #4 0xE3A07004	
svc 0 0xEF000000	
_exit:	
mov r7, #1	; 0xE3A07001
svc 0	; 0xEF000000
is_prime:	
mov r4, #2	; 0xE3A04002
cmp r0, #1	; 0xE3500001
ble prime_exit	; 0x02FFFFFFD
cmp r0, #2	; 0xE3500002
beq prime_exit	; 0x01A0000A
check_divisor:	
mov r5, r0	; 0xE1A05000
mov r6, #0	; 0xE3A06000
mov r7, #0	; 0xE3A07000
divide_loop:	
add r6, r6, #1	; 0xE2866001
sub r5, r5, r4	; 0xE2454004
cmp r5, #0	; 0xE3500000
blt divide_exit	; 0x08FFFFFF9
cmp r4, r5	; 0xE1540005
bge divide_loop	; 0x1AFFFFFF8

divide_exit:

sub r6, r6, #1 ; 0xE2466001

mov r0, r6 ; 0xE1A00006

bx lr ; 0xE12FFF1E

prime_exit:

moveq r0, #1 ; 0x03A00001

movne r0, #0 ; 0x01A00000

bx lr ; 0xE12FFF1E

mark_prime:

ldr r5, =prime_flag ; 0xE59F501C

ldr r6, =msg_len ; 0xE59F6020

ldr r7, [r5, r6] ; 0xE5977000

add r7, r7, #1 ; 0xE2877001

str r7, [r5, r6] ; 0xE5877000

ldr r7, =msg_len ; 0xE59F7024

ldr r8, [r5, r7] ; 0xE5978000

cmp r8, r3 ; 0xE1530000

bge continue_check ; 0x3A000002

str r3, [r5, r7] ; 0xE5878000

continue_check:

add r6, r6, #4 ; 0xE2866004

mov r7, #0 ; 0xE3A07000

str r7, [r5, r6] ; 0xE5877000

b check_prime ; 0xEAFFFFED

.data

msg: .asciz "Prime numbers:\n" ; 0x5062696D65206E75

msg_len: .word 25 ; 0x00000019